

Extraits de code commentés

Six extraits représentatifs de l'application « Petit Rayon de Soleil », illustrant l'authentification, le contrôle d'accès par rôle, la sécurité des requêtes et la conformité RGPD. Chaque extrait est reproduit fidèlement depuis le code source, avec son emplacement exact.

Fichiers `server.js`, `auth.js` **Stack** Node.js + Express + SQLite (node:sqlite)

Sécurité `bcrypt` · `tokens` · requêtes paramétrées

01 Inscription & connexion

server.js : 30-60

server.js

```
30 // POST /api/register – inscription. Body: { name, email, password }
31 app.post('/api/register', (req, res) => {
32   const { name, email, password } = req.body || {};
33   if (!name || !email || !password) {
34     return res.status(400).json({ error: 'Nom, email et mot de passe sont requis.' });
35   }
36   if (password.length < 6) {
37     return res.status(400).json({ error: 'Le mot de passe doit faire au moins 6 caract'
38   }
39   const existing = db.prepare('SELECT id FROM users WHERE email = ?').get(email);
40   if (existing) return res.status(409).json({ error: 'Cet email est déjà utilisé.' });
41
42   const info = db
43     .prepare('INSERT INTO users (name, email, password_hash) VALUES (?, ?, ?)')
44     .run(name, email, auth.hashPassword(password));
45
46   const token = auth.createSession(info.lastInsertRowid);
47   res.status(201).json({ token, user: { id: info.lastInsertRowid, name, email, role: '
48 });
49
50 // POST /api/login – connexion. Body: { email, password }
51 app.post('/api/login', (req, res) => {
52   const { email, password } = req.body || {};
53   const user = db.prepare('SELECT * FROM users WHERE email = ?').get(email);
54   // Message volontairement générique pour ne pas révéler si l'email existe.
55   if (!user || !auth.checkPassword(password, user.password_hash)) {
56     return res.status(401).json({ error: 'Email ou mot de passe incorrect.' });
57   }
58   const token = auth.createSession(user.id);
59   res.json({ token, user: { id: user.id, name: user.name, email: user.email, role: use
60 });
```

À RETENIR Validation des entrées, unicité de l'email (409), mot de passe **jamais stocké en clair** (hashPassword). À la connexion, message d'erreur **générique** : on ne révèle pas si l'email existe, pour ne pas aider un attaquant.

02 Middleware requireAuth

auth.js : 61-67

auth.js

```
61  /** Middleware : exige un utilisateur connecté. */
62  function requireAuth(req, res, next) {
63      const user = userFromToken(tokenFromReq(req));
64      if (!user) return res.status(401).json({ error: 'Non authentifié' });
65      req.user = user;
66      next();
67  }
```

À RETENIR Composant **réutilisable** placé devant chaque route sensible. Il lit le token (Bearer), retrouve l'utilisateur en base, et refuse l'accès (401) si le token est absent ou invalide. En cas de succès, l'utilisateur est attaché à req.user pour la suite.

03 Contrôle administrateur requireAdmin

auth.js : 69-76

auth.js

```
69  /** Middleware : exige un utilisateur connecté ET administrateur. */
70  function requireAdmin(req, res, next) {
71      const user = userFromToken(tokenFromReq(req));
72      if (!user) return res.status(401).json({ error: 'Non authentifié' });
73      if (user.role !== 'admin') return res.status(403).json({ error: 'Accès réservé à l'admin' });
74      req.user = user;
75      next();
76  }
```

À RETENIR Contrôle d'accès **par rôle**. Deux niveaux distincts : non connecté → 401 ; connecté mais pas administrateur → 403 (interdit). C'est ce middleware qui protège toutes les routes /api/admin/* de modération.

04 Requête SQL paramétrée (anti-injection)

auth.js : 19-33

auth.js

```
19  /** Hache un mot de passe en clair. */
20  function hashPassword(plain) {
21      return bcrypt.hashSync(plain, 10);
22  }
23
24  /** Vérifie un mot de passe en clair contre son hash stocké. */
25  function checkPassword(plain, hash) {
26      return bcrypt.compareSync(plain, hash);
27  }
28
29  /** Crée une session pour un utilisateur et renvoie le token. */
30  function createSession(userId) {
31      const token = genToken();
32      db.prepare('INSERT INTO sessions (token, user_id) VALUES (?, ?)').run(token, userId);
33      return token;
```

À RETENIR Toutes les requêtes utilisent des **paramètres préparés** (?) : les valeurs (token , userId ...) sont transmises séparément du SQL et ne peuvent pas être interprétées comme du code → **protection contre l'injection SQL**. Le token de session est de plus généré de façon imprévisible (crypto.randomBytes).

05 Suppression du compte (RGPD)

server.js : 85-90

server.js

```
85  // DELETE /api/me – suppression du compte et de toutes ses données (droit RGPD).
86  // Les favoris et sessions sont supprimés en cascade (ON DELETE CASCADE).
87  app.delete('/api/me', auth.requireAuth, (req, res) => {
88      db.prepare('DELETE FROM users WHERE id = ?').run(req.user.id);
89      res.json({ ok: true, message: 'Compte et données personnelles supprimés.' });
90  });
```

À RETENIR Droit à l'effacement (RGPD) : la suppression de l'utilisateur entraîne, **en cascade** (ON DELETE CASCADE défini au niveau de la base), la suppression de ses favoris et de ses sessions. Aucune donnée personnelle ne subsiste. Route protégée par requireAuth : on ne peut supprimer que son propre compte (req.user.id).

06 Ajout & retrait de favoris

server.js : 148-168

server.js

```
148 // POST /api/favoris – ajouter un message en favori. Body: { message_id }
149 app.post('/api/favoris', auth.requireAuth, (req, res) => {
150   const { message_id } = req.body || {};
151   const msg = db.prepare('SELECT id FROM messages WHERE id = ? AND approved = 1').get(
152     if (!msg) return res.status(404).json({ error: 'Message introuvable.' });
153   // INSERT OR IGNORE : pas d'erreur si le favori existe déjà.
154   db.prepare('INSERT OR IGNORE INTO favoris (user_id, message_id) VALUES (?, ?)').run(
155     req.user.id,
156     message_id
157   );
158   res.status(201).json({ ok: true });
159 });
160
161 // DELETE /api/favoris/:id – retirer un message des favoris.
162 app.delete('/api/favoris/:id', auth.requireAuth, (req, res) => {
163   db.prepare('DELETE FROM favoris WHERE user_id = ? AND message_id = ?').run(
164     req.user.id,
165     req.params.id
166   );
167   res.json({ ok: true });
168 });
```

À RETENIR Relation **N,N** (table de liaison favoris). L'ajout vérifie que le message existe et est approuvé, et utilise `INSERT OR IGNORE` pour éviter un doublon. Ajout comme retrait sont **filtrés sur** `req.user.id` : un utilisateur n'agit que sur ses propres favoris.

Extraits reproduits fidèlement depuis le code source de « Petit Rayon de Soleil » · Titre professionnel CDA. Numéros de ligne conformes aux fichiers `server.js` et `auth.js`.